

AC1 Trabalho Myvg

Aluno Daniel Santos 170070, Aluno Bernardo Gamula 158440

1 Objectivo

Este trabalho foi integralmente realizado pelos alunos Daniel Santos (170070) e Bernardo Gamula (158440) sem recurso a ferramentas de inteligência artificial.

O programa lê o ficheiro `.myvg`, e interpreta os comandos `p`, `l` e `r` para criar a imagem ascii pretendida.

Linhas vazias e linhas que comecam por `#` sao ignoradas.

2 Rotinas

- `sign`, `absv`: auxiliares para o algoritmo das linhas.
- `point`: poe um `#` na posicao (x,y). Se cair fora da imagem nao faz nada.
- `line`: algoritmo de Bresenham.
- `rect`: faz 4 chamadas ao `line`.
- `initimg`: pede memoria ao `sbrk` e enche tudo de espacos.
- `readln`, `nextc`, `peekc`, `skipsp`, `pint`: o parser, le linha a linha.
- `readf`: abre o ficheiro, le, e chama o parser em loop.
- `prting`: imprime a imagem no ecrã.
- `save`: escreve a imagem no ficheiro de saida.

3 Notas

Leitura Ler 1 linha de cada vez. `readln` le byte a byte com a `ecall` do `read` (`a2=1`) ate ao `\n`, e mete a linha num buffer (`lbuf`). Depois o `peekc/nextc` andam com um indice (`lpos`) por cima do `lbuf`, por isso o `peek` apenas le sem consumir.

Sem `mul/div` O enunciado diz que sao valorizados os trabalhos que nao usam `mul/div/rem`. Multiplicacao foi substituida por shifts:

- `Mul var * 10 -> pint: val*10 = (val<<3) + (val<<1).`
- Index array de pointers (4b): `slli x, 2.`
- O `2*e` do Bresenham: `slli x, 1.`

Ecalls Codigo foi feito para o RARS, portanto usa-se a tabela de `ecalls` do programa:

```

1024: open
63:  read
64:  write
57:  close
9:    sbrk
10:  exit
4:    print_str
11:  print_char
8:    read_str

```

Nota: \n no fim do nome. O `read_string` do RARS deixa \n no fim do que o utilizador escreve, isso faz com q o open nunca encontre o ficheiro. Procuramos primeiro \n no nome, depois \0.

4 Bresenham

Fluxograma dado no enunciado:

```

dx = abs(x2 - x1)
dy = -abs(y2 - y1)
sx = sign(x2 - x1)
sy = sign(y2 - y1)
e  = dx + dy

```

```

loop:
    point(x1, y1)
    if x1 == x2 and y1 == y2: stop
    e2 = 2 * e
    if e2 >= dy: e += dy; x1 += sx
    if e2 <= dx: e += dx; y1 += sy

```

Em asm o estado e' guardado em `s0..s9` (`x1, y1, x2, y2, dx, dy, sx, sy, e, e2`), com push/pop.

5 Exemplo

Input (`test.myvg`): (Input: "C:/..path../test.myvg")

```

10 10
p 0 0
r 7 7 9 9
l 0 9 9 0

```

Output ascii (`out.txt`):

```

#           #
           #
           #
           #
           #
           #
           #
#           ###
#           # #
#           ###

```

6 Observacoes

A parte mais complexa neste trabalho foi o parser. Le 1 linha para o buffer, vai lendo char a char com um indice, o peek e next lêem e avancam nesse indice. O algoritmo das linhas so com inteiros funciona como esperado e nao precisa de nenhuma virgula flutuante.